

ADC Configuration

1. Introduction

The Texas Instruments C2000 family of microcontrollers is widely used in real-time control applications, such as motor drives, renewable energy, automotive systems, and industrial automation. The Analog-to-Digital Converter (ADC) is a critical peripheral, enabling the MCU to convert analog signals (e.g., voltage, current, temperature) into digital values. This project demonstrates sampling four ADC channels (two each from ADCA and ADCC) using the LAUNCHXL-F2800137, with results stored in global variables and displayed in Code Composer Studio (CCS). The setup uses a potentiometer for testing and serves as a foundation for sensor interfacing.

2. Theory of ADC in C2000

The TMS320F2800137 features four 12/16-bit SAR ADC modules (ADCA, ADCB, ADCC, ADCD) with up to 21 external channels (64-pin PM package). This project uses 12-bit mode (0.8mV/step at 3.3V). Key concepts:

Resolution: Defines voltage step size ($3.3V / 4096 \approx 0.8mV$ for 12-bit).

Start of Conversion (SOC): Each ADC has multiple SOC's to select channels and triggers.

Interrupts: Triggered on conversion completion (polled in this project).

Result Registers: Store digital values per SOC.

Project Setup: Configures ADCA and ADCC, each with SOC0 and SOC1, for four simultaneous inputs.

3. Project Objective

Configure ADCA and ADCC modules.

Sample two channels per module (total four channels).

Store results in global variables.

Display results in CCS Expressions window.

Provide a scalable framework for sensor interfacing.

4. Hardware Requirements and Implementation

Components

TI LAUNCHXL-F2800137 LaunchPad.

10k Ω potentiometer (linear taper).

Breadboard and jumper wires.

USB Type-C cable.

Optional: Multimeter for verification.

Hardware Setup

1. Connect Power:

Use USB Type-C to power the board (5V, converted to 3.3V).

Ensure jumpers JP1/JP2 are installed; S3 set to UP-UP (emulation mode).

Verify LEDs LD1 (3.3V) and LD2 (debugger).

2. ADC Pin Connections:

ADCA SOC0 (ADCINA0/ADCINC15): BoosterPack Pin 44 (J3, position 4).

ADCA SOC1 (ADCINA1): BoosterPack Pin 62 (J7, position 2).

ADCC SOC0 (ADCINC0/ADCINA11): BoosterPack Pin 5 (J1, position 5).

ADCC SOC1 (ADCINC1/ADCINA12): BoosterPack Pin 4 (J1, position 4).

GND: BoosterPack Pin 20 (J2, position 10).

3.3V: BoosterPack Pin 3 (J1, position 3).

3. Potentiometer Wiring:

For each channel (e.g., ADCINA0):

Outer terminal 1 to GND (Pin 20).

Outer terminal 2 to 3.3V (Pin 3).

Wiper to ADC pin (e.g., Pin 44).

Repeat for other channels with separate potentiometers if needed.

Ensure input voltage stays within 0-3.3V; add 0.1μF capacitor for noise reduction if necessary.

4. Safety:

Use single-ended inputs (default for 12-bit).

Verify voltages with a multimeter.

Unplug USB before wiring to avoid shorts.

5. Software Requirements

Code Composer Studio (CCS): TI's IDE for building/flashing.

C2000Ware: Provides DriverLib, device support (F280013x), and examples.

DriverLib: Abstracts register-level programming with APIs (e.g., `ADC\_forceMultipleSOC()`).

6. Detailed Code Explanation

Code Purpose

Configures ADCA/ADCC to sample two channels each, storing results for debugging.

Key Components

Includes: `driverlib.h`, `device.h`, `board.h`.

Globals: `uint16\_t myADC0Result0`, `myADC0Result1`, `myADC1Result0`, `myADC1Result1` (12-bit results).

Main Flow:

Initialize system, GPIO, interrupts, and board.

Enable global interrupts and real-time mode.

Infinite loop: Trigger SOC, poll for completion, read/store results.

Full Code

```
#include "driverlib.h"
#include "device.h"
#include "board.h"

// Globals
uint16_t myADC0Result0;
uint16_t myADC0Result1;
uint16_t myADC1Result0;
uint16_t myADC1Result1;

void main(void)
{
    Device_init();
    Device_initGPIO();
    Interrupt_initModule();
```

```

Interrupt_initVectorTable();

Board_init();


EINT;

ERTM;

while(1)
{
    ADC_forceMultipleSOC(myADC0_BASE, (ADC_FORCE_SOC0 | ADC_FORCE_SOC1));
    while(ADC_getInterruptStatus(myADC0_BASE, ADC_INT_NUMBER1) == false) {}
    ADC_clearInterruptStatus(myADC0_BASE, ADC_INT_NUMBER1);


    ADC_forceMultipleSOC(myADC1_BASE, (ADC_FORCE_SOC0 | ADC_FORCE_SOC1));
    while(ADC_getInterruptStatus(myADC1_BASE, ADC_INT_NUMBER1) == false) {}
    ADC_clearInterruptStatus(myADC1_BASE, ADC_INT_NUMBER1);


    myADC0Result0 = ADC_readResult(ADCARESULT_BASE, ADC_SOC_NUMBER0);
    myADC0Result1 = ADC_readResult(ADCARESULT_BASE, ADC_SOC_NUMBER1);
    myADC1Result0 = ADC_readResult(ADCCRESULT_BASE, ADC_SOC_NUMBER0);
    myADC1Result1 = ADC_readResult(ADCCRESULT_BASE, ADC_SOC_NUMBER1);


    ESTOP0;
}
}

```

Working

1. Initialization:

`Device\_init()`: Sets clock (100 MHz), flash, peripherals.

`Device\_initGPIO()`: Configures pins for ADC (e.g., Pin 44 as ADCINA0).

`Board\_init()`: Sets 12-bit mode, internal 3.3V reference.

`EINT`/`ERTM`: Enables interrupts and real-time debug.

2. Sampling Loop:

Triggers SOC0/SOC1 on ADCA (`myADC0\_BASE`) and ADCC (`myADC1\_BASE`).

Polls interrupt flags (INT1) for completion.

Reads results into globals (0-4095).

ESTOP0` allows debug halts.

3. Key Notes:

Polling is CPU-intensive; replace with interrupts for efficiency.

Voltage = (result * 3.3) / 4096.

7. Testing Procedure

- 1. Connect hardware as described.
- 2. Build/flash code in CCS (F280013x target).
- 3. Open CCS Expressions window; add globals (`myADC0Result0`, etc.).
- 4. Rotate potentiometer; observe values:
 - 0V → ~0
 - 1.65V → ~2048
 - 3.3V → ~4095
- 5. Troubleshoot: Check pin mux, power, noise.

8. Results & Observations

Voltage (V) Expected ADC Value (12-bit)		
----- -----		
0.0	0	
0.5	~620	
1.0	~1240	
1.5	~1860	
2.0	~2480	
2.5	~3100	
3.0	~3720	
3.3	4095	

Observation: Linear response validates ADC setup.

9. Applications

Motor Control: Measure currents/voltages.

Temperature Monitoring: Multiple sensors.

Smart Agriculture: Soil moisture, light sensors.

Power Electronics: Voltage monitoring in converters.

10. Conclusion & Future Scope

Achievements: Successfully sampled four channels; results viewable in CCS.

Future Enhancements:

Use DMA for CPU-free transfers.

Implement ISRs for real-time response.

Calibrate ADC for accuracy.

Add ADCB/ADCD for more channels.

Integrate sensor fusion.

11. Key Takeaways

1. **ADC Basics:** 12-bit SAR, 0.8mV/step, SOC-driven.
2. **Project:** Samples four channels via ADCA/ADCC.
3. **Hardware:** LAUNCHXL-F2800137, potentiometer on pins like 44.
4. **Software:** DriverLib simplifies coding; polling-based.
5. **Testing:** Linear 0-4095 output.
6. **Applications:** Scalable for control systems.
7. **Future:** Interrupts, DMA, calibration.

Line-by-Line Code Explanation

Below is the full code with a detailed explanation of each line.

```
#include "driverlib.h"
```

- Includes the DriverLib header for peripheral APIs.
 - Provides functions like ADC_forceMultipleSOC() and ADC_readResult(). Part of C2000Ware, ensures portability across C2000 devices (e.g., F2800137).
-

```
#include "device.h"
```

- Includes device-specific definitions for the TMS320F2800137.
 - Defines constants (e.g., DEVICE_LSPCLK_FREQ, ADCA_BASE) for clock and register addresses.
-

```
#include "board.h"
```

- Includes board-specific initialization for LAUNCHXL-F2800137.
 - Defines Board_init() to configure ADC settings (12-bit, 3.3V reference) and pin muxing.
-

```
// Globals
```

```
uint16_t myADC0Result0;
```

```
uint16_t myADC0Result1;
```

```
uint16_t myADC1Result0;
```

```
uint16_t myADC1Result1;
```

- Declares four 16-bit unsigned integers to store ADC results
 - myADC0Result0, myADC0Result1: Results for ADCA SOC0 (e.g., ADCINA0, Pin 44) and SOC1 (e.g., ADCINA1, Pin 62).
 - myADC1Result0, myADC1Result1: Results for ADCC SOC0 (e.g., ADCINC0, Pin 5) and SOC1 (e.g., ADCINC1, Pin 4).
 - 12-bit results (0-4095, 0-3.3V) fit in uint16_t. Global for CCS Expressions monitoring.
-

```
void main(void)
```

- **Purpose:** Defines the program's entry point.
 - **Details:** Starts execution after MCU reset, initializes system, and runs the sampling loop.
-

Device_init();

- Initializes MCU core components.
 - Sets system clock (100 MHz for F2800137), flash, and peripheral clocks. Enables modules and clears watchdog timers.
-

Device_initGPIO();

- Initializes GPIO settings.
 - Configures pins for analog input (e.g., ADCINA0). Sets correct muxing to enable ADC functionality.
-

Interrupt_initModule();

- Initializes the interrupt controller.
 - Configures the Peripheral Interrupt Expansion (PIE) module. Prepares for interrupts, though polling is used here.
-

Interrupt_initVectorTable();

- Sets up the interrupt vector table.
 - Maps interrupt handlers to PIE vectors. Not used directly (no ISR), but required for system integrity.
-

Board_init();

- Performs board-specific initialization.
 - Configures ADCA/ADCC for 12-bit single-ended mode, internal 3.3V reference, and assigns SOC channels to pins. Defined in board.c.
-

EINT;

- Enables global interrupts.
 - Sets INTM = 0. Not needed for polling but included for compatibility with interrupt-based extensions.
-

ERTM;

- Enables real-time debugging mode.
- Allows CCS to halt execution without watchdog issues, facilitating live variable inspection.

while(1)

Starts an infinite loop for continuous ADC sampling.

Ensures real-time monitoring of analog inputs.

```
ADC_forceMultipleSOC(myADC0_BASE, (ADC_FORCE_SOC0 | ADC_FORCE_SOC1));
```

- Triggers Start of Conversion (SOC) for SOC0 and SOC1 on ADCA.
 - myADC0_BASE: ADCA base address.
 - ADC_FORCE_SOC0 | ADC_FORCE_SOC1: Triggers both SOC0s in burst mode.
 - Starts conversion on assigned channels (e.g., ADCINA0, ADCINA1).
-

```
while(ADC_getInterruptStatus(myADC0_BASE, ADC_INT_NUMBER1) == false) {}
```

- Polls ADCA's INT1 flag for conversion completion.
 - Waits until SOC0 and SOC1 conversions finish (INT1 tied to SOC1 in burst mode). CPU-intensive polling.
-

```
ADC_clearInterruptStatus(myADC0_BASE, ADC_INT_NUMBER1);
```

- Clears ADCA's INT1 flag.
 - Resets flag for the next conversion cycle.
-

```
ADC_forceMultipleSOC(myADC1_BASE, (ADC_FORCE_SOC0 | ADC_FORCE_SOC1));
```

- Triggers SOC0 and SOC1 on ADCC.
 - Similar to ADCA, starts conversions on ADCC channels (e.g., ADCINC0, ADCINC1).
-

```
while(ADC_getInterruptStatus(myADC1_BASE, ADC_INT_NUMBER1) == false) {}
```

- Polls ADCC's INT1 flag.
 - Waits for ADCC conversions to complete.
-

```
ADC_clearInterruptStatus(myADC1_BASE, ADC_INT_NUMBER1);
```

- Clears ADCC's INT1 flag.
 - Prepares ADCC for the next cycle.
-

```
myADC0Result0 = ADC_readResult(ADCARERESULT_BASE, ADC_SOC_NUMBER0);
```

- Reads ADCA SOC0 result.
 - Stores 12-bit value (0-4095) from ADCINA0 (e.g., Pin 44) in myADC0Result0.
-

```
myADC0Result1 = ADC_readResult(ADCARERESULT_BASE, ADC_SOC_NUMBER1);
```

- Reads ADCA SOC1 result.
 - Stores result from ADCINA1 (e.g., Pin 62) in myADC0Result1.
-

```
myADC1Result0 = ADC_readResult(ADCCRESULT_BASE, ADC_SOC_NUMBER0);
```

- Reads ADCC SOC0 result.
 - Stores result from ADCINC0 (e.g., Pin 5) in myADC1Result0.
-

```
myADC1Result1 = ADC_readResult(ADCCRESULT_BASE, ADC_SOC_NUMBER1);
```

- Reads ADCC SOC1 result.
 - Stores result from ADCINC1 (e.g., Pin 4) in myADC1Result1.
-

```
ESTOP0;
```

- Inserts a debug breakpoint.
 - Halts execution in CCS for variable inspection (e.g., myADC0Result0 in Expressions window). Ignored in standalone mode.
-

```
}
```

```
}
```

- Closes the infinite loop and main function.
- Ensures continuous sampling.